```
  key ←
treap.root.key
  #3
  remove(treap,
key)                                                    #4
  return
key
          #5
```

#1 Method `top` takes a treap instance and returns its top-priority element, unless the treap is empty.
#2 If the treap was empty, we need to throw an error.
#3 Store the key at the top root of the treap.
#4 Remove the top key from the treap.
#5 Return the key.

Besides validating the treap's status, checking that it's not empty, we just need to retrieve the key stored in the root, and then remove it from the treap.

Similarly, if we need to update the priority associated with a key, we can follow the same logic as for plain heaps, bubbling up the updated node (when increasing priority, or pushing down, when we lower priority): the only difference is that instead of just swapping nodes we need to perform rotations to move the updated node. Implementation of this method is left as an exercise (or you can check it out on the book's repo).

3.3.7 Min, Max

The last methods left to implement in our API are `min` and `max`, returning the minimum and maximum key stored in the treap.